

Project: Hostel Life Utility Manager App Name: HostelHub Document: Master AI Frontend Design Specification Purpose: Single source of truth for all AI coding agents working on the frontend.

1. CRITICAL INSTRUCTION FOR ALL AI AGENTS

You are an AI coding agent assisting one of three HUMAN TEAM MEMBERS working on the SAME HostelHub application.

The application must look and feel like it was created by ONE professional frontend team, not three independent developers.

This document is the SINGLE SOURCE OF TRUTH for the HostelHub frontend design and development rules.

Before writing, modifying, or generating ANY frontend UI code:

1. Read and follow this entire `.github/copilot-instructions.md` file.
2. Inspect the existing project structure.
3. Inspect the existing theme and shared components.
4. Reuse existing components whenever possible.
5. Follow the colors, typography, spacing, sizing, iconography, cards, navigation and interaction rules defined here.
6. Do NOT invent a new visual style for your feature.
7. Do NOT introduce new colors unless explicitly approved by the project owner.
8. Do NOT introduce a different font.
9. Do NOT create duplicate versions of existing shared components.
10. Do NOT redesign components already implemented by another team member.
11. Do NOT modify another member's feature unless it is required for integration.
12. Keep all screens visually consistent with the reference screenshots supplied by the team.
13. Use mock/local data for the frontend prototype unless backend integration has explicitly been requested.
14. Keep the UI architecture ready for future API/backend integration.
15. Do not add unnecessary packages or dependencies.
16. Do not change the global theme to make one particular page look better.
17. If a requirement conflicts with this document, follow this document unless the project owner explicitly changes the design system.
18. Before finishing, inspect the affected screens and verify that they visually belong to the same application.
19. Do not modify unrelated screens, components, layouts, navigation, or design systems while implementing a requested feature.
20. Preserve all existing functionality unless the requested task explicitly requires changing it.

MOST IMPORTANT RULE

DO NOT ALTER THE EXISTING APP'S SIZE, COLOUR, INTERFACE, OR UI DESIGN.

The AI agent MUST preserve the established HostelHub visual identity.

DO NOT independently change:

- screen dimensions or intended responsive behavior
- layout structure
- component sizes
- spacing
- padding
- margins
- typography
- font family
- font sizes
- font weights
- colors
- backgrounds
- borders
- border radius
- shadows
- icons
- icon sizes
- navigation structure
- header structure
- bottom navigation
- card design
- button design
- interaction patterns
- animations
- overall visual hierarchy

unless the project owner explicitly requests that specific change.

Use the existing design system and shared components first.

If an existing component, screen, or design already matches the project specification, DO NOT redesign it.

When adding a new feature, make the new feature visually belong to the existing application WITHOUT changing the established design of existing features.

The goal is to EXTEND the existing HostelHub interface, NOT redesign it.

ABSOLUTE DESIGN PRESERVATION RULE

The AI agent must NEVER change the existing app interface simply because it believes another design would be:

- more modern
- more attractive
- cleaner
- easier to implement
- more responsive
- more professional
- more visually appealing

The project owner's defined interface takes priority over the AI agent's personal design preferences. If the AI agent believes a UI change is necessary for functionality, accessibility, responsiveness, or technical compatibility, it must report the issue to the human team member BEFORE making a visual redesign.

No silent UI redesigns are allowed.

INSTRUCTION FILE PROTECTION

AI agents MUST NOT:

- modify this instruction file
- delete this instruction file
- rename this instruction file
- overwrite this instruction file
- remove rules from this instruction file
- weaken rules from this instruction file
- replace these instructions with their own preferences

unless the project owner explicitly instructs the agent to modify the instructions.

If a task conflicts with these instructions:

1. Do NOT silently change the instructions.
2. Do NOT remove the conflicting rule.
3. Identify the conflict.
4. Report the conflict to the human team member.
5. Wait for explicit direction before changing the rules.

The AI agent must treat this file as the authoritative source for HostelHub frontend design decisions.

1. REFERENCE SCREENS

The following screenshots are the visual references for the entire application:

- Reference 1: Home page — upper section

- Reference 2: Home page — lower section
- Reference 3: Schedule page
- Reference 4: Hamburger menu

These references define the application's:

- visual hierarchy
- colors
- typography
- spacing
- card style
- navigation
- icon style
- rounded corners
- shadows
- section layout
- overall visual personality

All future screens must look like they belong to these screens.

Do NOT copy the screenshots as static images.

Recreate the UI using real frontend components.

1. PRODUCT STRUCTURE

The application contains the following sections:

HOSTEL LIFE UTILITY MANAGER | |— ENTRY | |— Splash | |— Onboarding | |— Login |
|— MAIN APP | |— Home | |— Schedule | |— Alerts | |— Profile | |— HOSTEL SERVICES
| |— Room | |— Mess | |— Laundry | |— Repair | |— Events | |— Resources | |—
SECONDARY

```
|— Hamburger Menu
  |— Settings
  |— Notification Preferences
  |— Help & Support
  |— Report a Problem
  |— About
```

There is NO fee/payment feature in this application.

Do not add:

- Fees
- Payments

- Payment history
 - UPI
 - Wallet
 - Billing
 - Transactions
 - Fee reminders
-

1. DESIGN PHILOSOPHY

HostelHub should feel:

- modern
- clean
- friendly
- minimal
- student-oriented
- organized
- lightweight
- easy to scan
- visually calm

The design should NOT feel:

- corporate
- overly dark
- overly colorful
- cluttered
- gaming-style
- heavily gradient-based
- overly animated

The interface should use mostly:

White + very light backgrounds + dark text + blue primary accents + soft service colors.

1. PRIMARY COLOR SYSTEM

Use these colors consistently.

Primary Blue

Primary: #326DF0 Primary Dark: #1D43B6 Primary Light: #94B9F2 Deep Navy: #061258

Blue is the main brand/action color.

Use it for:

- HostelHub brand accent
- active bottom navigation
- primary buttons
- selected tabs
- selected filters
- important links
- primary icons
- interactive elements
- section "View All" links
- active states

Do NOT use blue for every card.

1. BASE COLORS

Background: #FAFCFF Surface: #FFFFFF Surface Soft: #F4F8FF Border: #DDE7F5

Primary Text: #061258 Secondary Text: #5B6B85 Tertiary Text: #8A98AD Disabled Text: #B6C0CF

Divider: #E6EDF7

The overall application should remain predominantly light.

Cards should generally use white or extremely soft tinted backgrounds.

1. SEMANTIC COLORS

Use soft semantic colors rather than highly saturated backgrounds.

Success / Green

Green: #26A66A Green Light: #EAF8F1

Use for:

- completed
- available
- events
- positive updates
- successful states

Warning / Orange

Orange: #F2A51A Orange Light: #FFF5DE

Use for:

- upcoming

- scheduled
- mess
- reminders
- attention states

Error / Red

Red: #E34D59 Red Light: #FFF0F1

Use for:

- urgent notices
- errors
- logout/destructive actions
- unresolved issues

Blue

Blue: #326DF0 Blue Light: #EEF5FF

Use for:

- informational states
- laundry-related secondary UI where appropriate
- neutral information

1. SERVICE COLOR LANGUAGE

Each hostel service may have a soft visual identity.

Room

Icon: #1D43B6 Background: #EEF4FF

Mess

Icon: #326DF0 Background: #F0F6FF

Laundry

Icon: #061258 Background: #EAF1FF

Repair

Icon: #E04B55 Background: #FFF0F1

Events

Icon: #35A86F Background: #EDF9F2

Resources

Icon: #22A6A6 Background: #EDFAFA

These colors should be soft and coordinated.

Do not introduce random colors for individual service pages.

1. TYPOGRAPHY

Use:

Font Family: Inter

If Inter is not currently available, use a clean system sans-serif fallback.

Do NOT use decorative fonts.

Typography scale

Display / Hero: 28 px Weight: 700

Page Title: 24 px Weight: 700

Section Heading: 20 px Weight: 700

Card Heading: 16 px Weight: 600

Body: 15 px Weight: 400

Body Medium: 15 px Weight: 500

Small / Supporting: 13 px Weight: 400

Caption: 12 px Weight: 500

Navigation Label: 13 px Weight: 500

Typography rules

Headings should be bold but not excessively heavy.

Body text should remain easy to read.

Secondary information should use the secondary text color rather than shrinking text too much.

Do not use more than necessary font weights.

1. GENERAL SPACING SYSTEM

Use a consistent 4-point spacing system.

4 px = XS 8 px = SM 12 px = MD 16 px = LG 20 px = XL 24 px = XXL 32 px = XXXL

Preferred page horizontal padding:

16–20 px

Use consistent vertical spacing between sections.

Do not randomly use values such as:

13 px 17 px 23 px 27 px

unless there is a strong layout reason.

1. BORDER RADIUS

Use soft rounded corners throughout the application.

Small components: 8 px Buttons: 12 px Standard cards: 16 px Large cards: 18–20 px Bottom sheets: 24 px

The Home page cards in particular should have a soft rounded appearance similar to the reference screenshots.

Avoid sharp rectangular cards.

1. CARD SYSTEM

Cards are a major part of HostelHub.

Default card:

Background: #FFFFFF Border: 1 px #DDE7F5 Radius: 16 px Padding: 16 px

Shadow should be extremely subtle.

Avoid heavy shadows.

Preferred appearance:

clean soft slightly elevated minimal

Cards should never look like floating 3D objects.

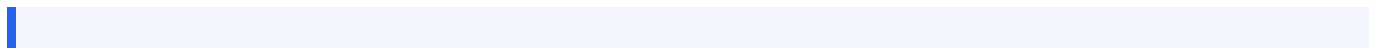
1. SERVICE CARDS

Service cards should contain:

[Icon]

Service Name

Short description



Example:

Use the service's predefined color.

Do not use a different card design for every service.

1. HEADER / APP BAR

The main application header follows the reference design.

Home:

≡ HostelHub  

Your hostel, organized.

Other major pages can use:

≡ Page Title Action

Header rules:

- Keep it clean.
- Use generous vertical spacing.
- Hamburger icon on the left where applicable.
- Page title is bold.
- Actions appear on the right.
- Do not create oversized app bars.
- Do not use colored app bars unless specifically required.

1. HOSTELHUB BRANDING

The brand is:

HostelHub

"Hostel" should be dark.

"Hub" should use the primary blue.

Example:

HostelHub

Subtitle:

Your hostel, organized.

The subtitle uses secondary text.

1. HOME PAGE DESIGN

The Home page is the main dashboard.

The layout should follow the supplied screenshots.

Order:

Header ↓ Greeting + Date ↓ Weather Card ↓ Room Snapshot ↓ For You ↓ Quick Access ↓ Today's Schedule
↓ Today's Menu ↓ Active Repair ↓ Important Notice ↓ Upcoming Event ↓ Bottom Navigation

The page should be vertically scrollable.

1. HOME — GREETING

Example:

Good Morning, Sid 🤖

Today

Use:

Greeting: 24 px / 700 Date: 15 px / 400

The greeting should be visually prominent.

Do not make the date larger than the greeting.

1. WEATHER CARD

The weather card appears toward the top-right area of the Home page on larger layouts.

Example:

☀️ 29°C Chennai

It should be a compact, soft card.

The weather card is informational and should not dominate the dashboard.

For the frontend prototype, mock weather data may be used.

1. ROOM SNAPSHOT

The Room Snapshot card is directly below the greeting area.

Example:

Block B • Room 304

3rd Floor 2 Roommates

View Room >

Use a soft blue-tinted background.

The Room Snapshot should immediately communicate:

- Block
- Room
- Floor
- Roommates

Clicking it navigates to the Room page.

1. FOR YOU SECTION

This is one of the most important sections of the application.
It represents the personalized/predictive part of HostelHub.

Example cards:

Laundry Slot Tomorrow at 10:00 AM

Repair Scheduled Fan repair Tomorrow, 2–4 PM

Mess Update Dinner menu has changed

Cards can have different soft service colors.

The section should contain:

For You View All

"View All" uses primary blue.

1. QUICK ACCESS

Quick Access contains the six hostel services:
Room Mess Laundry Repair Events Resources

Use a consistent grid.

Each item must have:

- service icon
- service name

- short description
- navigation affordance where appropriate

Do not add Payments.

1. TODAY'S SCHEDULE

The Home page should show a compact schedule preview.

Example:

Today's Schedule View Schedule

12:30 PM 🍽️ Lunch 05:00 PM 🧺 Laundry 06:00 PM 🏀 Sports 07:30 PM 🍽️ Dinner

Use service colors consistently.

The full schedule exists on the Schedule page.

1. TODAY'S MENU

Display:

Breakfast Lunch Snacks Dinner

Each meal can contain:

- icon
- meal name
- menu
- optional timing

Use the Mess service color.

1. ACTIVE REPAIR

Display the current repair issue.

Example:

Fan not working Room 304

Tomorrow, 2:00 PM – 4:00 PM Technician: Arun

Reported ✓ Assigned ✓ Scheduled ● Completed ○

Use a compact status tracker.

Statuses must be visually understandable.

1. IMPORTANT NOTICE

Notices should use a soft red/pink background when urgent.

Example:

Important Notice

Water supply will be interrupted tomorrow from 9:00 AM to 11:00 AM.

[View All >](#)

Do not use bright red backgrounds.

1. UPCOMING EVENTS

Example:

Upcoming Event

Hostel Cricket Cup 20 Aug • 5:00 PM • Ground 2

[View Events >](#)

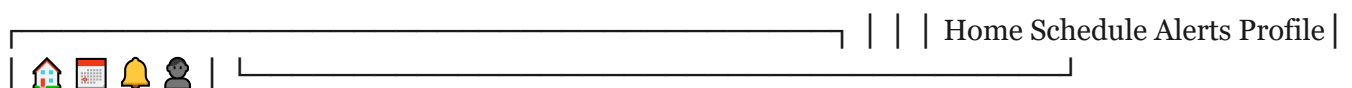
Use the Events green color language.

1. BOTTOM NAVIGATION

The primary bottom navigation contains exactly four items:

Home Schedule Alerts Profile

Visual structure:



Rules:

- 4 items only.
- Active item uses primary blue.
- Inactive items use neutral gray.
- Icons above labels.
- Label size approximately 13 px.
- Keep navigation visually lightweight.
- Alerts may display an unread badge.
- Do not add Room/Mess/Laundry to bottom navigation.

1. SCHEDULE PAGE

The Schedule page must follow Reference 3.

Top:

≡ Schedule Calendar

Then:

Today | Tomorrow | This Week

Selected tab:

Primary Blue White text Rounded corners

Then filters:

All Mess Laundry Repair Events

Then the chronological timeline.

Example:

08:00 AM | • Breakfast | 12:30 PM | • Lunch | 05:00 PM | • Laundry Slot | 06:00 PM | • Badminton |
07:30 PM | • Dinner | 09:00 PM | • Fan Repair

Use service-specific colors for timeline indicators and cards.

1. SCHEDULE CARDS

Every schedule item should contain:

Time Icon Title Category Location Start – End time Status

Example:

Breakfast Mess • Main Dining Hall 08:00 AM – 09:00 AM Upcoming

Cards should use the same global card design.

1. HAMBURGER MENU

Follow Reference 4 closely.

The hamburger menu should open as a side drawer.

It contains:

HostelHub Your hostel, organized.

Profile section

ACCOUNT My Profile My Hostel Details

ACTIVITY Notifications All Notices My Schedule

PREFERENCES Settings Notification Preferences

SUPPORT Help & Support Report a Problem About HostelHub

Logout

Do not put payment-related items in this menu.

1. HAMBURGER MENU VISUAL STYLE

The drawer should:

- occupy approximately 80–90% of the screen width
- have a white background
- have rounded top-right/bottom-right corners where appropriate
- have a dimmed overlay over the remaining screen
- use primary blue for important icons
- use subtle section dividers
- have generous vertical spacing
- contain a clear close button

Logout should use the red semantic color.

1. PROFILE PAGE

Profile is one of the four primary navigation destinations.

Display:

Profile Photo

Student Name Student ID

Course Batch

Block Room Floor

Then relevant account/preferences sections.

Use the same typography, cards and spacing as the rest of the application.

Do not create a completely different profile design.

1. ALERTS PAGE

Alerts are different from the hamburger menu.

The Bell icon and bottom navigation Alerts both lead to the Alerts experience.

Possible categories:

All Smart Alerts Notices Updates

Example:

Laundry Reminder Your laundry slot is tomorrow at 10:00 AM.

Repair Scheduled Fan repair is scheduled tomorrow, 2–4 PM.

Mess Update Dinner menu has changed.

Unread alerts should have a clear but subtle indicator.

Use semantic/service colors.

1. SMART ALERTS

The application's differentiating feature is proactive information.

The frontend should support messages such as:

Your laundry slot is tomorrow.

Your mess menu has changed.

Your room's maintenance request is scheduled.

Your upcoming event starts in 2 hours.

The frontend should be designed so that these messages can later be generated by a backend/prediction system.

Do NOT hard-code the concept of AI into every screen.

The UI should simply present useful personalized information.

1. ENTRY SCREENS

Splash

Minimal.

HostelHub Your hostel, organized.

Use the primary brand colors.

Do not overload the splash screen.

Onboarding

Explain the three main benefits:

Everything in one place Stay updated Know what's next

Use the same design system.

Login

Clean form.

Use:

- Email/Student ID
- Password
- Login button

Use the same primary button component as the rest of the application.

1. BUTTON SYSTEM

Primary button:

Background: Primary Blue Text: White Height: approximately 48 px Radius: 12 px

Secondary button:

Background: Primary Light Text: Primary Blue

Outlined button:

Background: Transparent Border: Primary Blue Text: Primary Blue

Do not create different button styles for different pages.

1. INPUT FIELDS

Inputs should have:

Height: approximately 48–52 px Radius: 12 px Border: #DDE7F5 Background: #FFFFFF

Focused state:

Border: Primary Blue

Error state:

Border: Red Supporting text: Red

1. ICONOGRAPHY

Use one consistent icon family throughout the application.

Preferred style:

- clean
- rounded/outlined where appropriate
- simple
- easily recognizable
- consistent stroke weight

Do not mix several unrelated icon styles.

Do not use emoji as the actual UI icon system unless the existing implementation specifically requires it.

Emoji shown in reference content such as greetings are content, not the required icon library.

1. RESPONSIVE DESIGN

The UI must work on different mobile screen sizes.

Do not hard-code the entire interface around a single screenshot resolution.

Use:

- responsive constraints
- flexible widths
- safe areas
- scrolling
- adaptive layouts
- appropriate minimum/maximum widths

The reference screenshots are visual references, not fixed pixel canvases.

1. SCROLLING

Home, Schedule, Alerts, Profile and service pages should support vertical scrolling where content exceeds the viewport.

Never allow content to disappear underneath the bottom navigation.

Respect device safe areas.

1. LOADING STATES

Every data-driven section should be architected so a loading state can be added.

Use:

- skeletons
- subtle progress indicators
- placeholders

Do not use large blocking loading screens unnecessarily.

1. EMPTY STATES

Every list-based feature should have an empty state.

Examples:

No upcoming repairs.

No alerts right now.

No laundry bookings.

No upcoming events.

Empty states must follow the same typography and card system.

1. ERROR STATES

Errors should be clear and friendly.

Example:

Something went wrong.

We couldn't load your schedule.

[Try Again]

Use the existing button and card components.

1. ANIMATIONS

Animations should be subtle.

Allowed:

- page transitions
- drawer slide animation
- card fade/slide
- tab selection
- button feedback
- loading shimmer

Avoid:

- excessive bouncing
- flashy transitions
- unnecessary particle effects

- large animations
- gaming-style effects

Animation should improve usability, not distract from the content.

1. ACCESSIBILITY

The frontend should support:

- readable contrast
- sufficiently large tap targets
- semantic labels
- accessible icons
- screen-reader-friendly buttons
- scalable text where technically appropriate

Never communicate important information through color alone.

Example:

Do not show only:

GREEN = Completed

Also show:

Completed ✓

1. COMPONENT REUSE

Before creating a component, search the existing project.

If an equivalent component exists, reuse it.

Examples of shared components:

AppHeader AppBottomNavigation AppCard ServiceCard SectionHeader PrimaryButton
SecondaryButton StatusBadge ScheduleCard AlertCard EmptyState LoadingState ProfileHeader

Do not create:

LaundryCard MessCard RepairCard EventsCard

with four completely different visual systems if they can use the same reusable component architecture.

Service-specific content can be configured through properties.

1. THEME ARCHITECTURE

Centralize design values.

Do not scatter colors throughout the code.

Preferred structure:

theme/ |—— colors |—— typography |—— spacing |—— dimensions |—— shadows |—— app_theme

Example:

AppColors.primary AppColors.primaryDark AppColors.primaryLight AppColors.deepNavy
AppColors.background AppColors.surface AppColors.textPrimary AppColors.textSecondary
AppColors.success AppColors.warning AppColors.error

Similarly:

AppSpacing.sm AppSpacing.md AppSpacing.lg

and:

AppRadius.sm AppRadius.md AppRadius.lg

The exact implementation can follow the project's framework.

1. DO NOT HARDCODE DESIGN VALUES REPEATEDLY

Avoid:

color: Color(...) padding: EdgeInsets.all(...) borderRadius: BorderRadius.circular(...)

repeated throughout many screens.

Instead use the centralized theme.

This ensures that all three team members produce consistent UI.

1. MOCK DATA

Until backend integration begins, use realistic mock data.

Example student:

Name: Sid Student ID: STU1024 Block: B Room: 304 Floor: 3rd Floor Roommates: 2

Example services:

Laundry Slot: Tomorrow, 10:00 AM

Repair: Fan not working Tomorrow, 2:00 PM – 4:00 PM

Mess: Dinner menu updated

Event: Hostel Cricket Cup 20 Aug, 5:00 PM

MOCK DATE RULE

When displaying dates in mock data:

- Do not treat example dates in this document as permanent production dates.
- Prefer relative dates such as "Today", "Tomorrow", "Friday", or "In 2 days" where appropriate.
- If an absolute date is required for a UI demonstration, use a date appropriate to the current demo scenario.
- Mock dates must remain centralized in the mock-data layer rather than being duplicated across UI screens.

Mock data should be centralized rather than duplicated across screens.

1. FUTURE BACKEND COMPATIBILITY

The current frontend is a prototype.

Do not tightly couple the UI to hard-coded data.

Prefer:

UI ↓ Model ↓ Repository / Service layer ↓ Mock data currently ↓ API/backend later

The UI should not need to be completely rewritten when the backend is connected.

1. TEAM OWNERSHIP

The frontend is divided between three team members.

Member 1 — Core + Home

Responsible for:

Entry |— Splash |— Onboarding |— Login

Core |— Theme |— Shared Components |— Navigation

Home |— Dashboard

Member 1 owns the global design system.

Other members should consume the shared components rather than redesigning them.

Member 2 — Hostel Services

Responsible for:

Room Mess Laundry Repair Events Resources

Member 2 must use the shared:

- theme

- cards
 - buttons
 - headers
 - typography
 - spacing
 - service color system
-

Member 3 — Personal + Communication

Responsible for:

Schedule Alerts Profile Hamburger Menu Settings Notification Preferences Help & Support Report a Problem About

Member 3 must use the same global design system.

1. FILE OWNERSHIP RULE

Do not unnecessarily modify another member's files.

Before changing shared files:

1. Check whether the change is actually necessary.
2. Discuss it with the team.
3. Preserve existing behavior.
4. Keep the design system backward-compatible.

Shared files are especially sensitive:

theme navigation shared widgets global constants main application shell

1. GIT / COLLABORATION RULE

Before starting work:

1. Check the current Git branch.
2. Pull the latest changes for the working branch when appropriate.
3. Confirm that the working tree does not contain unexpected uncommitted changes.
4. Work only on the assigned feature.

During development:

- Do not overwrite another team member's work.
- Do not modify unrelated features.
- Keep commits focused on the feature being developed.

After completing a meaningful feature:

Save ↓ Test ↓ Review ↓ Commit ↓ Push

Do not overwrite another member's changes.

If a merge conflict occurs:

- 1. Do NOT blindly choose "Accept All".
- 2. Inspect both versions.
- 3. Understand what each change is intended to accomplish.
- 4. Preserve both intended features where possible.
- 5. Resolve the conflict carefully.
- 6. Test the affected functionality after resolving it.

1. AI AGENT WORKFLOW

Every AI agent must follow this sequence:

READ DESIGN RULES

↓

INSPECT PROJECT

↓

INSPECT EXISTING COMPONENTS

↓

IDENTIFY REUSABLE COMPONENTS

↓

PLAN FEATURE

↓

IMPLEMENT

↓

TEST

↓

CHECK DESIGN CONSISTENCY

↓

REPORT CHANGES

Do not immediately start creating files without inspecting the existing project.

1. AI AGENT RESPONSE REQUIREMENT

After completing a task, report:

1. Files created
2. Files modified
3. Components reused
4. Components created
5. Navigation added/changed
6. Mock data added
7. Dependencies added, if any
8. Any unresolved issues

Do not claim that something works unless it has actually been checked.

55A. AI CHANGE SAFETY RULES

Before modifying existing code, the AI agent must determine whether the change is:

1. Required for the requested feature.
2. Required for integration.
3. Required to fix a genuine bug.
4. Unrelated to the requested task.

If the change is unrelated, do NOT make it.

EXISTING FUNCTIONALITY

AI agents must preserve:

- existing navigation
- existing working screens
- existing shared components
- existing theme behavior

- existing feature behavior
- existing mock-data relationships

unless the task explicitly requires a change.

SHARED FILES

Treat the following as high-risk shared files:

- theme
- navigation
- main application shell
- shared widgets/components
- global constants
- routing
- shared models
- shared services

Before modifying a high-risk shared file:

1. Determine why the modification is necessary.
2. Make the smallest reasonable change.
3. Preserve backward compatibility.
4. Check that existing screens still work.
5. Report the modification after completing the task.

DEPENDENCY SAFETY

Before adding a package:

1. Check whether the required functionality already exists in the project.
2. Check whether an existing dependency can provide the functionality.
3. Add a new dependency only when it provides a clear benefit.
4. Report every newly added dependency.

Do not add packages simply because they make implementation slightly easier.

SCOPE CONTROL

When asked to implement one feature:

Implement the requested feature first.

Do not automatically:

- redesign the Home page
- restructure the entire project
- replace the navigation system
- replace the theme
- rewrite unrelated components

- refactor unrelated code
- introduce a new architecture

unless explicitly requested or required for safe integration.

1. STRICT "DO NOT" LIST

AI agents MUST NOT:

- Add payments
 - Add fees
 - Add billing
 - Add wallet functionality
 - Add unnecessary features
 - Change the primary color
 - Change the font family
 - create a second design system
 - create random card styles
 - create random button styles
 - use random gradients
 - use excessive shadows
 - use excessive animations
 - mix unrelated icon libraries
 - redesign another member's page
 - remove existing functionality without approval
 - change navigation without approval
 - create duplicate shared components
 - hard-code repeated design values
 - use fixed screen-sized layouts
 - add backend logic unless requested
 - add external services unless requested
-

1. QUALITY CHECK BEFORE FINISHING

Before declaring a frontend task complete, verify:

Visual

- ☐ Colors match the design system.
- ☐ Typography matches the design system.
- ☐ Spacing follows the 4-point system.
- ☐ Cards use the standard radius.

- ☐ Buttons use the standard styles.
- ☐ Icons are consistent.
- ☐ Navigation matches the application.
- ☐ The page looks like it belongs to HostelHub.

Functional

- ☐ Navigation works.
- ☐ Buttons work.
- ☐ Scrolling works.
- ☐ Back navigation works.
- ☐ Empty states exist where needed.
- ☐ Loading states are considered.
- ☐ No content is hidden behind navigation.

Code

- ☐ Existing components were reused.
- ☐ No unnecessary duplicate components were created.
- ☐ No unnecessary dependencies were added.
- ☐ Design values are centralized.
- ☐ Mock data is separated from UI where possible.
- ☐ Other features were not unnecessarily modified.

1. FINAL DESIGN PRINCIPLE

The goal is NOT:

«"Make every screen identical."»

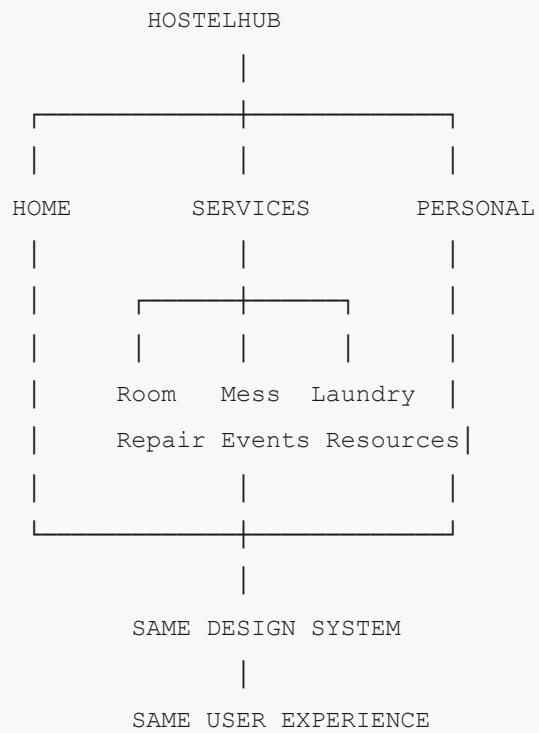
The goal is:

«Make every screen feel like it belongs to the same product.»

Different services may have different semantic colors and content, but they must share:

Typography + Spacing + Cards + Buttons + Navigation + Icon style + Header style + Interaction patterns
+ Design tokens

The final application should feel like:



All three AI agents must treat this file as the single source of truth for the HostelHub frontend design.